

Reading Exercise 02 — Finding main()

Every program has a beginning. This exercise finds it, follows it two steps, and stops.

What this exercise trains

The ability to find a program's entry point and follow its first few calls is a fundamental navigation skill. It does not require understanding the program. It requires only knowing where to look.

In most programs, `main()` is not where the interesting work happens. It is the airport: transit, not destination. But understanding the airport tells you what is adjacent to it and how to get anywhere else.

Step 1 — Find main() [5 minutes]

Locate `main()` in `practice/tcc/tcc.c`:

```
grep -n "^int main" practice/tcc/tcc.c
```

Open the file at that line. Read `main()` end to end. It is short.

Write down:

- The line number where `main()` begins
 - How many lines long is it?
 - How many function calls does it make?
 - What is the first function call, and what do you guess it does from the name alone?
-

Step 2 — One level down [10 minutes]

Take the first function call that `main()` makes (not a standard library call — the first TCC-specific call). Find its definition.

```
grep -n "^tcc_\|^static.*tcc_\|^int tcc_\|^void tcc_" practice/tcc/libtcc.c  
| head -20
```

Or use:

```
grep -rn "^LIBTCCAPI\|^ST_FUNC\|^PUB_FUNC" practice/tcc/*.c | grep "function_name"
```

Read that function for 5 minutes. You will not understand all of it. Read for shape, not content.

Write down:

- The function name and file it lives in
- Approximately how many lines long is it?
- What is the first thing it does?
- What is the last thing it does?

Step 3 — One level further [10 minutes]

Take the first TCC-specific function call *inside the function from Step 2*. Find it. Read it for 5 minutes.

Stop after 5 minutes, whether or not you feel done.

Write down:

- The function name and file
- One thing you understood about what it does
- One thing you did not understand

Step 4 — The call tree [5 minutes]

Draw (on paper or in your notes) a simple call tree:

```
main()
  ■■■ [Step 2 function]
    ■■■ [Step 3 function]
```

You have traced three levels of a program that has many, many more. That is enough.

Write down: at Step 3, do you feel closer to understanding TCC or further? Why?

Step 5 — The surprise [3 minutes]

Return to `tcc.c` and read lines 22–29 again. Find the `#include "libtcc.c"` line.

TCC includes its own source files — not headers, the .c files themselves. This is unusual. Most programs compile each .c file separately and link the objects. TCC compiles everything as one translation unit.

Write down: what do you think the advantage of this approach is? What might be a disadvantage? (There is no trick — think it through.)

Debrief

You have now walked the call tree three levels deep. You started from a 432-line wrapper file and ended up somewhere in the middle of a function you don't yet understand.

That is normal. That is what reading code feels like at the start.

The next time you read TCC code, you will recognize some of the function names you saw today. Recognition accumulates. Comprehension follows recognition, not the other way around.

Next: RE-03 — Following a Token — tracking one C keyword through the entire compilation pipeline.